

# Fraud-Spike Detector

Razorpay AI Buildathon - Track 02: AI Risk Manager

## 30-second summary

Fraud-Spike Detector is an explainable transaction-risk system. It takes a transaction, estimates the probability of fraud, chooses a bounded action, tells the reviewer which features influenced the score, and stores the decision in an audit trail.

This is more than a model-only notebook:

Transaction -> Features -> ML risk score -> Safe action -> Explanation -> Audit log

The current prototype uses a controlled synthetic dataset. The pipeline is ready to be adapted to a public anonymized dataset or real merchant data.

## What problem does it solve?

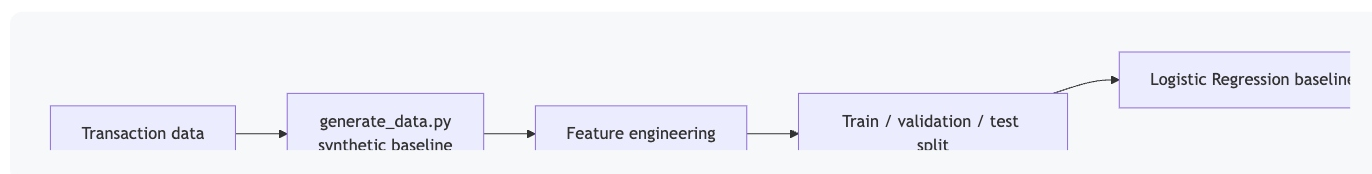
A payment system must catch fraud without unnecessarily blocking genuine customers. The detector looks at amount, time, transaction velocity, device novelty, distance from home, and the customer's recent average.

It balances two kinds of mistakes:

| Mistake        | Meaning                                 | Illustrative cost |
|----------------|-----------------------------------------|-------------------|
| False positive | Genuine customer is incorrectly stopped | INR 150           |
| False negative | Fraud transaction is allowed through    | INR 4,500         |

The cost assumptions are explicit and can be changed in `train_model.py`.

## Architecture



## Runtime request flow

```
POST /check-transaction
|
v
Validate input with Pydantic
|
v
Create derived features
(amount_to_avg_ratio, is_odd_hour)
|
v
XGBoost predicts fraud probability
|
+--> Compare score with thresholds
+--> SHAP calculates top reasons
+--> Save decision to SQLite
|
v
Return score + action + reasons + timestamp
```

## Risk actions

---

The API does not treat every suspicious transaction identically. It returns one of five bounded actions:

```
Low score      -> allow
Slight concern -> monitor
Medium risk    -> step_up_verification
High risk      -> manual_review
Very high risk -> block
```

This makes the project closer to a real risk-management workflow than a simple fraud classifier.

t-

## How the model is trained

---

`train_model.py` follows this process:

1. Load `transactions.csv`.
2. Add `amount_to_avg_ratio` and `is_odd_hour`.
3. Split data into training, validation, and final test sets.
4. Train a balanced Logistic Regression baseline.
5. Train an imbalance-aware XGBoost model.
6. Select the business-cost threshold using validation data only.
7. Report final metrics on the untouched test set.
8. Fit a SHAP explainer and save the model artifacts.

This separation prevents the final test set from influencing threshold selection.

## Results

---

Current results use 20,000 synthetic transactions with approximately 1.7% fraud. The final test set contains 2,000 transactions and was not used for threshold selection.

| Model                              | Precision | Recall | F1    | ROC-AUC | Estimated cost |
|------------------------------------|-----------|--------|-------|---------|----------------|
| Logistic Regression baseline       | 0.348     | 0.941  | 0.508 | -       | INR 18,000     |
| XGBoost at selected threshold 0.05 | 0.344     | 0.941  | 0.504 | 0.979   | INR 18,150     |
| XGBoost at default threshold 0.50  | 0.714     | 0.882  | 0.790 | 0.979   | INR 19,800     |

## How to read these results

- **Precision:** Of the transactions flagged as fraud, how many were actually fraud.
- **Recall:** Of all actual fraud transactions, how many the model caught.
- **F1:** Balance between precision and recall.
- **ROC-AUC:** How well the model separates fraud from legitimate transactions across thresholds.
- **Estimated cost:** False-positive cost plus false-negative cost.

The default threshold gives better precision and F1. The lower selected threshold prioritizes recall because missed fraud is assumed to be much more expensive than customer friction. The best choice depends on the merchant's real cost ratio.

## Setup and run

---

macOS/Linux:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt

# macOS only: required by XGBoost's native library
brew install libomp

python generate_data.py
python train_model.py
```

```
uvicorn app:app --reload
```

Open the interactive API documentation at <http://127.0.0.1:8000/docs>.

## Try the API

### Suspicious transaction

```
curl -X POST http://127.0.0.1:8000/check-transaction \
-H "Content-Type: application/json" \
-d '{
  "transaction_id": "demo_fraud_001",
  "amount": 320,
  "hour_of_day": 3,
  "txn_velocity_1h": 6,
  "is_new_device": 1,
  "distance_from_home_km": 480,
  "avg_txn_last_30d": 40
}'
```

Expected behavior: a high fraud score, usually `block`, and SHAP reasons such as high velocity, unusual distance, and a new device.

### Normal transaction

```
curl -X POST http://127.0.0.1:8000/check-transaction \
-H "Content-Type: application/json" \
-d '{
  "transaction_id": "demo_normal_001",
  "amount": 40,
  "hour_of_day": 14,
  "txn_velocity_1h": 0,
  "is_new_device": 0,
  "distance_from_home_km": 2,
  "avg_txn_last_30d": 45
}'
```

Expected behavior: a low fraud score and `allow`.

## API endpoints

| Endpoint                             | Purpose                                                |
|--------------------------------------|--------------------------------------------------------|
| <code>POST /check-transaction</code> | Score a transaction and return action plus explanation |

|                                 |                                      |
|---------------------------------|--------------------------------------|
| GET /audit-log                  | View recent decisions                |
| GET /audit-log/{transaction_id} | View one decision                    |
| GET /metrics                    | View training and evaluation metrics |
| GET /health                     | Check API and model status           |

Example response shape:

```
{
  "transaction_id": "demo_fraud_001",
  "decision": "block",
  "fraud_score": 1.0,
  "threshold_used": 0.05,
  "top_reasons": [
    {"feature": "txn_velocity_1h", "value": 6, "impact": 5.46},
    {"feature": "distance_from_home_km", "value": 480, "impact": 4.99},
    {"feature": "is_new_device", "value": 1, "impact": 0.68}
  ],
  "timestamp": "2026-08-22T08:48:43Z"
}
```

## Why this project is useful

- Handles severe class imbalance instead of relying on misleading accuracy.
- Compares a baseline with an improved tree-based model.
- Uses business cost, not only ML metrics, to discuss threshold selection.
- Explains every decision with the top three SHAP feature contributions.
- Records every decision, input, score, action, reason, and timestamp.
- Returns controlled 422 errors for invalid input and 503 when the model is missing.

## Limitations and next steps

The current version is intentionally a working prototype:

- The data is synthetic, not merchant production data.
- User-history values are currently supplied in the request; production should calculate them from a trusted store such as Redis or PostgreSQL.
- The system scores one transaction at a time and does not yet detect fraud rings.
- Cost assumptions are illustrative and must be replaced with merchant economics.
- A reviewer dashboard and confirmed-fraud feedback loop are natural next steps.

For a stronger production version the next architecture would be:

For a stronger production version, the next architecture would be:

```
Payment events -> History service -> Rules + ML model -> Risk action
|
+--> Reviewer dashboard
+--> Audit database
+--> Confirmed-fraud feedback
+--> Model monitoring and retraining
```

## Project files

| File                             | Responsibility                                            |
|----------------------------------|-----------------------------------------------------------|
| <code>generate_data.py</code>    | Creates the reproducible synthetic dataset                |
| <code>train_model.py</code>      | Feature engineering, training, evaluation, SHAP artifacts |
| <code>app.py</code>              | FastAPI scoring service, explanations, and audit logging  |
| <code>requirements.txt</code>    | Python dependencies                                       |
| <code>transactions.csv</code>    | Generated training data, reproducible from the script     |
| <code>model.pkl</code>           | Saved model, SHAP explainer, feature list, and threshold  |
| <code>metrics_report.json</code> | Evaluation metrics and threshold sweep                    |
| <code>audit_log.db</code>        | SQLite audit trail created by the API                     |

## Competition positioning

This is not only a fraud classifier. It is an explainable, cost-aware, and auditable transaction-risk decision system that balances fraud loss against genuine-customer friction.